1) Write a python program to implement RSA Algorithm.

Algorithm: RSA Encryption

1. Start
2. Select two large prime numbers p and q.
3. Compute:
   - $n = p \times q$
   - $\varphi(n) = (p - 1)(q - 1)$
4. Choose an integer e such that:
   - $1 < e < \varphi(n)$
   - $\gcd(e, \varphi(n)) = 1$
5. Compute private key d such that:
   - $d \equiv e^{-1} \pmod{\varphi(n)}$
6. Public key = (e, n)
7. Private key = (d, n)
8. Convert plaintext message M into integer form.
9. Encrypt using:
   - C = M^e mod n
10. Output ciphertext C.
11. Stop

Algorithm: RSA Decryption

1. Start
2. Take ciphertext C.
3. Use private key (d, n).
4. Decrypt using:
   - M = C^d mod n
5. Convert numeric message back to plaintext.
6. Display the original message.
7. Stop

Program:

```
import math
# Function to check coprime
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
# Function to find modular inverse
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None
```

```
# Step 1: Input two prime numbers
p = int(input("Enter a prime number p: "))
q = int(input("Enter a prime number q: "))
# Step 2: Compute n and phi(n)
n = p * q
phi = (p - 1) * (q - 1)
# Step 3: Choose public key e
e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1
# Step 4: Compute private key d
d = mod_inverse(e, phi)
# Display keys
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
# Step 5: Input message
message = int(input("Enter message (numeric value): "))
# Encryption
ciphertext = pow(message, e, n)
print("Encrypted Message:", ciphertext)
# Decryption
decrypted_message = pow(ciphertext, d, n)
print("Decrypted Message:", decrypted_message)
```

Output:

```
Enter a prime number p : 3
Enter a prime number q : 11
Enter message : 7
Public Key: (e = 3, n = 33)
Private Key: (d = 7, n = 33)
Encrypted Message: 13
Decrypted Message: 7
```